LAB MANUAL

K.R. MANGALAM UNIVERSITY

SOET

(School of Engineering & Technology)

B.TECH CSE (CORE)

Database & Management System

Submitted by-

Akansha

Roll no - 2401010195

Submitted to-

Miss Mansi Kajal

Experiment-1

Aim

To design an ER diagram for a College Database and convert it into relational tables.

Objectives

1. To identify the entities in the College Database.
2. To find the attributes of each entity.
3. To identify the relationships between entities.
4. To draw the ER diagram.
5. To convert the ER diagram into tables.

Entities and Attributes

STUDENT is the entity **and** USN (Primary Key), SName, Address, Phone, Gender are attributes.

SEMSEC is the entity **and** SSID (Primary Key)**,** Sem**,** Sec are attributes.

SUBJECT is the entity **and** Subcode (Primary Key)**,** Title**,** Sem**,** Credits are attributes.

Relationships

CLASS: Relationship between STUDENT and SEMSEC and it shows that a student belongs to a particular semester and section.
Type: Many-to-one (Many students belong to one semester and section)
IAMARKS: Relationship between STUDENT, SUBJECT, and SEMSEC and stores internal assessment marks of students

Attributes of IAMARKS are Test1, Test2, Test3, FinalIA

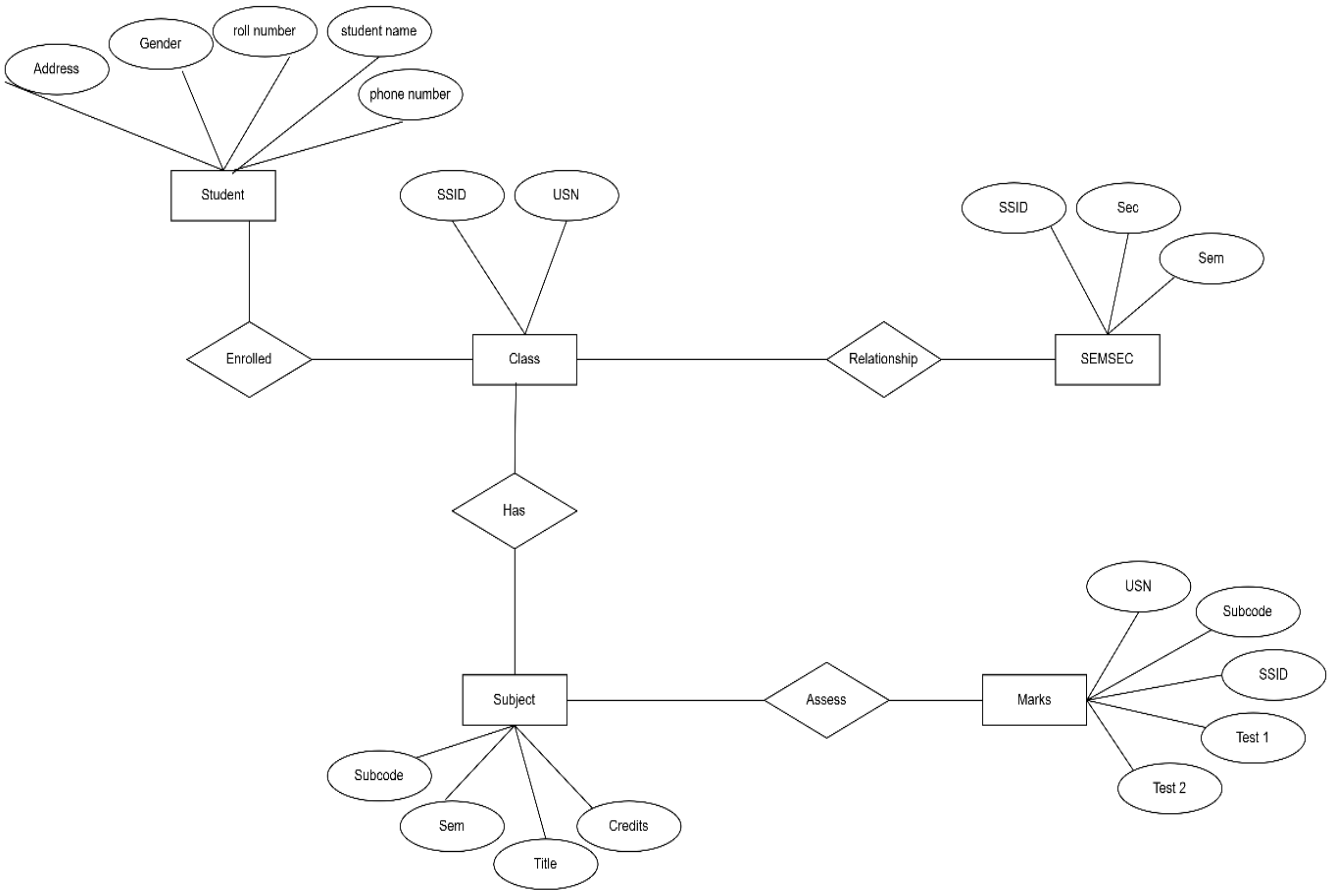
Type: Many-to-many relationship

Explanation:

STUDENT is an entity that stores student details such as USN, name, address, phone, and gender.
SEMSEC stores semester and section information using SSID, semester, and section.
SUBJECT contains subject details like subject code, title, semester, and credits.

The CLASS relationship shows that each student belongs to a particular semester and section. This is a many-to-one relationship between STUDENT and SEMSEC.

The IAMARKS relationship stores internal assessment marks of students for different subjects. It represents a many-to-many relationship between STUDENT and SUBJECT and includes attributes like Test1, Test2, Test3, and FinalIA. This relationship is many-to-many because a student can take many subjects, and each subject can be taken by many students


Address
Gender
roll number
student name
phone number
Student
SSID
USN
SSID
Sec
Sem
Enrolled
Class
Relationship
SEMSEC
Has
USN
Subcode
SSID
Test 1
Test 2
Subject
Assess
Marks
Subcode
Sem
Title
Credits

EXPERIMENT 2

Aim: To design an Entity–Relationship (ER) diagram for a company database system and define the relationship types between entities.

DESCRIPTION:

The COMPANY database contains information about employees, departments, projects, department locations and the working details of employees on projects.
The main entities in the database are EMPLOYEE, DEPARTMENT, PROJECT, DLOCATION and WORKS_ON.

ENTITIES :

1. EMPLOYEE**:** EMPLOYEE entity stores information about employees working in the company.
Attributes are :SSN (Primary Key), Name, Address, Sex, Salary, SuperSSN, DNo
SSN uniquely identifies each employee.
SuperSSN represents the supervisor of an employee (recursive relationship).
DNo is a foreign key which refers to the department in which the employee works.
2. DEPARTMENT: DEPARTMENT entity stores information about departments.
Attributes are: DNo (Primary Key), DName, MgrSSN, MgrStartDate
Each department is managed by one employee.
MgrSSN is a foreign key referring to EMPLOYEE.
3. PROJECT: PROJECT entity stores details of projects.
Attributes are: PNo (Primary Key), PName, PLocation, DNo
Each project is controlled by one department.
One department can control many projects.
4. DLOCATION: DLOCATION entity represents the different locations of a department.
Attributes: DNo, DLoc
A department can have multiple locations.
The primary key is the combination of DNo and DLoc.
5. WORKS_ON: WORKS_ON entity represents the relationship between EMPLOYEE and PROJECT.
Attributes: SSN, PNo, Hours
This resolves the many-to-many relationship between employees and projects.
The primary key is the combination of SSN and PNo.

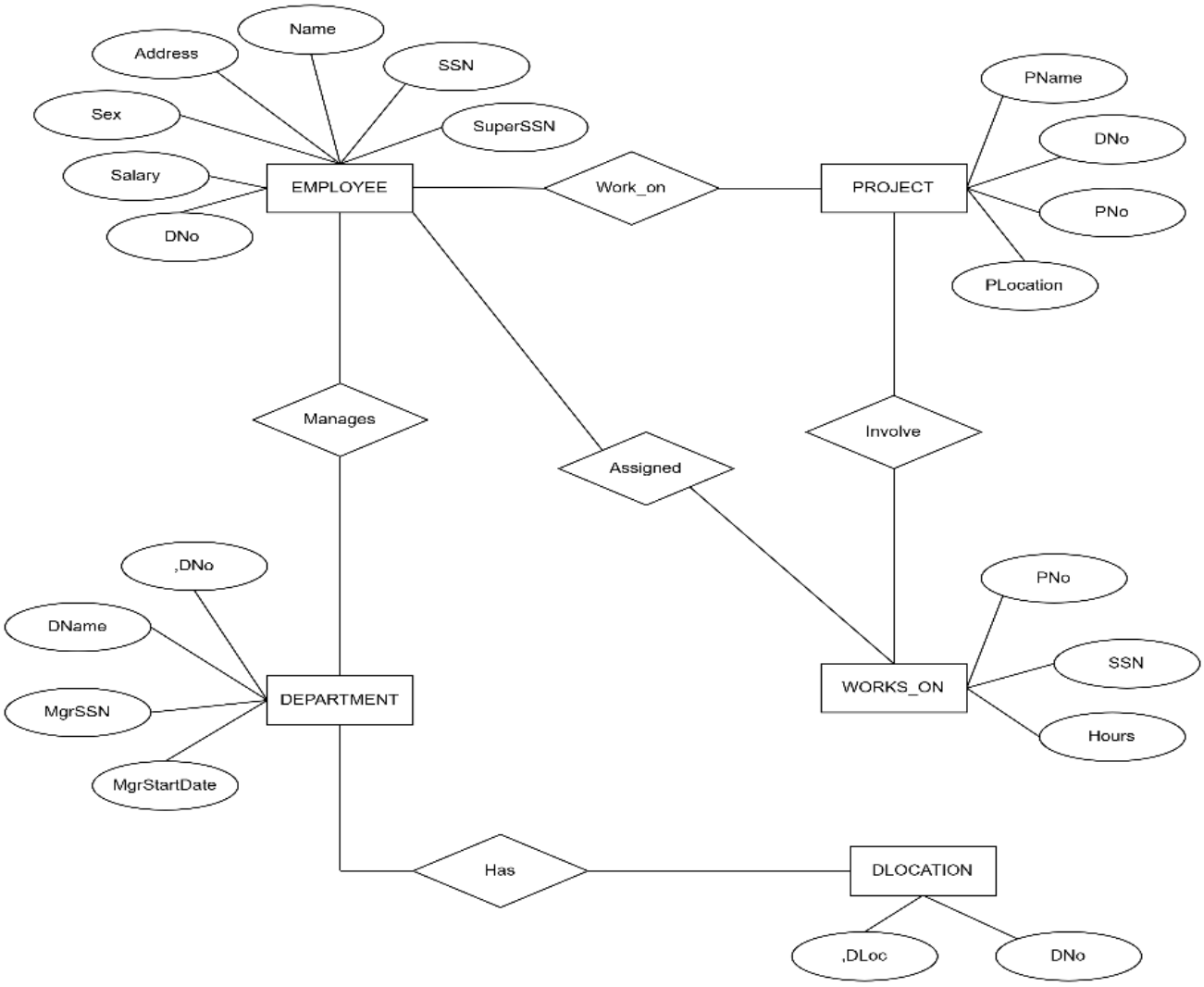
RELATIONSHIPS:

1. EMPLOYEE – WORKS_ON –PROJECT:
   An employee can work on many projects and a project can have many employees.
   This is a many-to-many relationship and is represented using WORKS_ON with attribute *Hours*.
2. EMPLOYEE – MANAGES – DEPARTMENT:
   An employee manages a department.
   Each department has one manager and an employee manages at most one department (1:1 relationship).

3. DEPARTMENT – ASSIGNED – PROJECT:
   A department controls many projects.
   Each project is controlled by only one department (1:N relationship).
4. DEPARTMENT – HAS – DLOCATION:
   A department can have multiple locations.
   This is a one-to-many (1:N) relationship.
5. EMPLOYEE – SUPERVISES – EMPLOYEE:
   An employee can supervise other employees.
   This is a recursive one-to-many (1:N) relationship.

EXPLANATION:

The ER diagram represents the COMPANY database.
EMPLOYEE stores employee details and each employee works in one department.
DEPARTMENT stores department details and is managed by an employee.
PROJECT stores project details and is controlled by a department.
WORKS_ON shows which employee works on which project and the number of hours.



EXPERIMENT :3

Database Schema for a customer-sale scenario Customer (Cust id :integer, cust_name :string)
Item (item_id: integer, item_name: string, price: integer)
Sale(bill_no:integer,bill_data:date,cust_id: integer, item_id: integer, qty_sold: integer)

For the above schema, perform the following—

1. Create the tables with the appropriate integrity constraints.

2. Insert around 10 records in each of the tables.

3. List all the bills for the current date with the customer names and item numbers.

4. List the total Bill details with the quantity sold, price of the item and the final amount.

5. List the details of the customer who have bought a product which has a price>200.

6. Give a count of how many products have been bought by each customer

7. Give a list of products bought by a customer having cust_id as 5.

8. List the item details which are sold as of today.

9. Create a view which lists out the bill_no, bill_date, cust_id, item_id, price, qty_sold, amount.

10. Create a view which lists the daily sales date wise for the last one week

CODE:

```
CREATE DATABASE akansha;
USE akansha;
CREATE TABLE Customer (
cust_id INT PRIMARY KEY,
cust_name VARCHAR (60) NOT NULL
);
CREATE TABLE Item (
item_id INT PRIMARY KEY,
item_name VARCHAR (50) NOT NULL,
price INT CHECK (price>0)
);
CREATE TABLE Sale (
bill_no INT PRIMARY KEY,
bill_date DATE NOT NULL,
cust_id INT,
item_id INT,
qty_sold INT NOT NULL CHECK (qty_sold >0),
FOREIGN KEY (cust_id) REFERENCES Customer(cust_id),
FOREIGN KEY (item_id) REFERENCES Item(item_id)
);
INSERT INTO Customer VALUES
(1, 'Akansha'), (2,'Dhruv'), (3, 'Neha'), (4,'Anil kumar'), (5, 'Deepmala');

INSERT INTO Item VALUES
```

```
(101, 'PEN' ,50),(102, 'Notebook', 120),(103, 'Bag', 700), (104, 'Watch', 1000), (105,'Shoes', 1500);

INSERT INTO Sale VALUES
(1, CURDATE (),1, 103, 1),
(2, CURDATE (), 2, 101, 5),
(3, CURDATE (), 5,104, 1),
(4, CURDATE (), 3, 105, 2),
(5, CURDATE (), 4, 102, 3);

SELECT s.bill_no, c.cust_name, i.item_name, s.qty_sold, i.price,
(s.qty_sold*i.price) AS final_amount

FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id;

SELECT DISTINCT c.cust_id, c.cust_name
FROM Sale s
JOIN Customer c ON s.cust_id = c.cust_id
JOIN Item i ON s.item_id = i.item_id
WHERE i.price >200;

SELECT c.cust_id, c.cust_name,
SUM (s.qty_sold) AS total_product_bought
FROM Sale s
JOIN Customer c ON s.cust_id =c.cust_id
GROUP BY c.cust_id, c.cust_name;

SELECT i.item_name, s.qty_sold
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.cust_id = 5;

SELECT DISTINCT i.*
FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE s.bill_date = CURDATE ();

CREATE VIEW Bill_Details AS
SELECT s.bill_no, s.bill_date, s.cust_id, s.item_id, i.price, s.qty_sold,(i.price*s.qty_sold) AS amount

FROM Sale s
JOIN Item i ON s.item_id = i.item_id;
SELECT *FROM Bill_Details;
CREATE VIEW Weekly_Sales AS
SELECT bill_date,
SUM(i.price*s.qty_sold) AS total_daily_sales
```

```
  FROM Sale s
JOIN Item i ON s.item_id = i.item_id
WHERE bill_date >= CURDATE ()- INTERVAL 7 DAY
GROUP BY bill_dates;

SELECT*FROM Weekly_Sales;
```

bill_no	cust_name	item_name	qty_sold	price	final_amount
1	Akansha	Bag	1	700	700
2	Dhruv	PEN	5	50	250
3	Deepmala	Watch	1	1000	1000
4	Neha	Shoes	2	1500	3000
5	Anil kumar	Notebook	3	120	360

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name
1	Akansha
5	Deepmala
3	Neha

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

cust_id	cust_name	total_product_bought
1	Akansha	1
2	Dhruv	5
5	Deepmala	1
3	Neha	2
4	Anil kumar	3

Result Grid | Filter Rows:

item_name	qty_sold
Watch	1

Result Grid | Filter Rows: | Export:

item_id	item_name	price
103	Bag	700
101	PEN	50
104	Watch	1000
105	Shoes	1500
102	Notebook	120

Field Types

#	Field	Schema	Table	Type	Character Set	Display Size	Precision	Scale
1	bill_no	akansha	bill_details	INT	binary	11	1	
2	bill_date	akansha	bill_details	DATE	utf8mb4	10	10	
3	cust_id	akansha	bill_details	INT	binary	11	1	
4	item_id	akansha	bill_details	INT	binary	11	3	
5	price	akansha	bill_details	INT	binary	11	4	
6	qty_sold	akansha	bill_details	INT	binary	11	1	
7	amount		bill_details	BIGINT	binary	21	4	

Form Editor | Field Types | Query Stats

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

bill_no	bill_date	cust_id	item_id	price	qty_sold	amount
1	2026-02-20	1	103	700	1	700
2	2026-02-20	2	101	50	5	250
3	2026-02-20	5	104	1000	1	1000
4	2026-02-20	3	105	1500	2	3000
5	2026-02-20	4	102	120	3	360

EXPERIMENT: 4

Consider the following table: Table: CLASS

ID	Name
1	Bravo
2	Alex
4	Cheng

Give the output of the following SQL script:

- INSERT INTO class VALUES (5,’Rahul’);
- COMMIT;
- UPDATE class SET name = ‘Abhijeet’ WHERE id= ‘5’;
- SAVEPOINT A;
- INSERT INTO class VALUES (6, ‘Chris’);
- SAVEPOINT B;
- INSERT INTO class VALUES (7, ‘Bravo’);
- SAVEPOINT C
- SELECT * FROM class;
- ROLLBACK TO B;
- SELECT * FROM class;
- ROLLBACK TO A;

CODE:

```
CREATE DATABASE assignment4;
USE assignment4;
CREATE TABLE Class (
ID INT PRIMARY KEY,
NAME VARCHAR (30)
);
INSERT INTO Class VALUES
(1,'Bravo'),(2,'Alex'),(3,'Cheng'),(4,'Hyimang'),(5,'Sowon'),(6,'Somang'),(7,'Hope');

INSERT INTO Class VALUES (8,'Rahul');
COMMIT;
UPDATE Class
SET NAME='Abhijeet'
WHERE ID =5;

SAVEPOINT A;
INSERT INTO Class VALUES (9,'Chris');
SAVEPOINT B;
INSERT INTO Class VALUES (10,'Bravo');
SAVEPOINT C;

SELECT*FROM Class;
ROLLBACK TO B;
SELECT*FROM Class;
ROLLBACK TO A;
```

Result Grid | Filter Rows: | Edit:

ID	NAME
1	Bravo
2	Alex
3	Cheng
4	Hyimang
5	Abhijeet
6	Somang
7	Hope
8	Rahul
9	Chris
10	Bravo
NULL	NULL